

ECLAT (Equivalence Class Clustering and Bottom-Up Lattice Traversal) Algorithm

ECLAT is a frequent itemset mining algorithm, similar to **Apriori** and **FP-Growth**.

The key difference is:

Algorithm	Data Format
Apriori	Horizontal (Transactions $\rightarrow$ Items)
FP-Growth	FP-Tree
ECLAT	Vertical (Items $\rightarrow$ Transaction IDs)

ECLAT finds frequent itemsets using **set intersections of Transaction IDs (TIDs)**.

Example

Suppose we have the following transactions:

Transaction ID	Items
T1	Bread, Milk
T2	Bread, Diaper, Beer
T3	Milk, Diaper, Beer
T4	Bread, Milk, Diaper, Beer
T5	Bread, Milk, Diaper

Assume:

Minimum Support = 3

Step 1: Convert to Vertical Format

Instead of storing transactions, ECLAT stores:

Item $\rightarrow$ List of Transaction IDs

Item	TID List
Bread	{T1,T2,T4,T5}
Milk	{T1,T3,T4,T5}
Diaper	{T2,T3,T4,T5}
Beer	{T2,T3,T4}

Support count:

Item	Support
Bread	4
Milk	4
Diaper	4
Beer	3

All are frequent because support ≥ 3 .

Step 2: Generate 2-Itemsets Using TID Intersections

{Bread, Milk}

Bread = {T1, T2, T4, T5}
Milk = {T1, T3, T4, T5}

Intersection:

{T1, T4, T5}

Support:

3

Frequent ✓

{Bread, Diaper}

Bread = {T1, T2, T4, T5}
Diaper = {T2, T3, T4, T5}

Intersection:

{T2, T4, T5}

Support:

3

Frequent ✓

{Bread, Beer}

Bread = {T1, T2, T4, T5}

Beer = {T2, T3, T4}

Intersection:

{T2, T4}

Support:

2

Not Frequent ✕

{Milk, Diaper}

Milk = {T1, T3, T4, T5}

Diaper = {T2, T3, T4, T5}

Intersection:

{T3, T4, T5}

Support:

3

Frequent ✓

{Milk, Beer}

Milk = {T1, T3, T4, T5}

Beer = {T2, T3, T4}

Intersection:

{T3, T4}

Support:

2

Not Frequent ✕

{Diaper, Beer}

Diaper = {T2, T3, T4, T5}

Beer = {T2, T3, T4}

Intersection:

{T2, T3, T4}

Support:

3

Frequent ✓

Frequent 2-Itemsets

{Bread, Milk}

{Bread, Diaper}

{Milk, Diaper}

{Diaper, Beer}

Step 3: Generate 3-Itemsets

Using previously frequent itemsets.

{Bread, Milk, Diaper}

Take:

{Bread, Milk} = {T1, T4, T5}

Intersect with Diaper:

{T1, T4, T5}

∩

{T2, T3, T4, T5}

=

{T4, T5}

Support:

2

Not Frequent ✕

{Milk, Diaper, Beer}

Milk = {T1, T3, T4, T5}
Diaper = {T2, T3, T4, T5}
Beer = {T2, T3, T4}

Intersection:

{T3, T4}

Support:

2

Not Frequent ✕

Final Frequent Itemsets

1-Itemsets

{Bread}
{Milk}
{Diaper}
{Beer}

2-Itemsets

{Bread, Milk}
{Bread, Diaper}
{Milk, Diaper}
{Diaper, Beer}

No frequent 3-itemsets exist for support = 3.

Visual Understanding

Apriori

Generate Candidates

↓

Scan Database

↓

Generate More Candidates

↓

Scan Again

Many scans.

ECLAT

Convert to TID Lists

↓

Intersect TID Sets

↓

Compute Support

↓

Generate Frequent Itemsets

Fewer database scans.

Why ECLAT Is Faster Than Apriori

Suppose:

Bread = {1,2,5,7,8}

Milk = {1,2,3,7,8}

Instead of rescanning the database, ECLAT simply computes:

{1,2,5,7,8}

∩

{1,2,3,7,8}

=

{1,2,7,8}

Support = 4

This makes ECLAT very efficient for dense datasets.

Apriori vs FP-Growth vs ECLAT

Feature	Apriori	FP-Growth	ECLAT
Candidate Generation	Yes	No	No
Database Scans	Many	2	1–2
Data Structure	Itemsets	FP-Tree	TID Sets
Speed	Slowest	Fast	Very Fast

Feature	Apriori	FP-Growth	ECLAT
Memory Usage	Medium	Low	Higher for large TID lists
Best For	Small datasets	Large datasets	Dense datasets